USBSID-Pico Midi manual

Author: LouD - generated on 2026-10-25

Table of Contents

Disclaimer . . . 3
Quick start . . . 3
Notes, velocity and aftertouch . . . 3
Channel routing . . . 4
Control Change reference . . . 4
- Voice waveform and envelope . . . 4
- Filter and volume . . . 5
- SID / voice selection and channel mode . . . 5
- Modulation and timing . . . 6
- Second LFO . . . 7
- Unison . . . 7
- Panic and reset . . . 7
- Cynthcart . . . 8
- Deprecated . . . 8
- Not wired . . . 8

Patches . . . 8
- Factory patches . . . 9
- Authoring your own patches over SysEx . . . 9

Arp tables . . . 10
- Using a table . . . 10
- Authoring tables over SysEx . . . 11

Saving your settings . . . 11
FMOpl . . . 12
- Targeting a channel at FMOpl . . . 12
- FMOpl patches . . . 12
- Live control . . . 13
- Known limitations . . . 13
- A note on tuning . . . 14

Development . . . 14
License . . . 14
- Software License ~ GNUv2 . . . 14
- Configtool Software License ~ EPLv2 . . . 14
- USBSID-Pico v1.5 PCB Hardware License ~ CC BY-NC-ND 4.0 . . . 14
- USBSID-Pico v1.3 PCB Hardware License ~ CC BY-SA 4.0 . . . 14

USBSID-Pico v1.0 PCB Hardware License ~ CC BY-SA 4.0 15
USBSID-Volume-daughterboard v1.0 PCB Hardware License ~ CC BY-SA 4.0 15

Disclaimer

I do this stuff in my free time for my enjoyment. Since I like to share my joy in creating this with everyone I try my best to provide a working PCB and Firmware. I am in no way an electronics engineer and can give ***no guarantees*** that this stuff does not break or damage your hardware, computer, phone, or whatever you try to hook it up to. Be sure to take great care when inserting any real MOS SID chips into the board. While everything has been tested with real chips, this is in no way a guarantee that nothing could go wrong. Use of this board and firmware at your own risk! I am in no way responsible for your damaged hardware. That being said, have fun!

Quick start

USBSID-Pico shows up as a class-compliant USB-MIDI device, no driver install needed on any platform. Point any MIDI application, DAW or live-coding environment (Strudel, VMPK, a hardware keyboard through a MIDI interface, …) at it and start playing.

Out of the box, without touching any configuration:

- **Channel 1** uses all voices on all configured SIDs, fully polyphonic (up to 12 voices on a 4-SID board). Plug in a keyboard and play.
- **Channels 2-5** get one SID chip each (channel 2 → SID 1, channel 3 → SID 2, through channel 5 → SID 4), 3-voice polyphonic within that chip. One instrument per channel, one channel per chip.
- **Channels 6-16** are disabled (no voice mask) until you assign them yourself.
- All channels default to a plain triangle wave with full sustain, a note is audible without sending a waveform or envelope CC first

If your board has an FMOpl-capable chip installed, see FMOpl below. It is a separate instrument with a smaller set of controls.

Notes, velocity and aftertouch

Standard MIDI Note On / Note Off (`0x90`/`0x80`) trigger and release voices. A Note On with velocity 0 is a Note Off, like most keyboards send under running status.

Velocity does nothing to the sound by default (a velocity curve cannot stomp on the channel volume CC). Turn `CC_VELM` on (see the table below) to let velocity scale the decay stage (higher velocity = punchier, shorter decay).

Channel Aftertouch (`0xD0`) and Polyphonic Key Pressure (`0xA0`, applied channel-wide) both drive the target `at_target` selects for that channel. Default is filter cutoff, a loaded patch can set channel volume or LFO depth (vibrato-on-pressure).

Pitch Bend (`0xE0`) bends all voices held on the channel live, using the bend range of the channel in semitones (2 by default or as set by the loaded patch, there is no CC for bend range).

Channel routing

Every MIDI channel (1-16) has independent routing, separate from the loaded patch:

- **Voice mask** ~ which of the voice slots (up to 12 on a 4-SID board, 3 per chip) the channel may use. Set up out of the box (see Quick start). Configured with the USBSID-Pico config tool, there is no MIDI CC for it.
- **SID / voice overrides** ~ `CC_SID1`-`CC_SID4` and `CC_VCE1`-`CC_VCE3` narrow the voice mask of a channel to one chip or one voice position, the configured mask stays untouched. Send any nonzero value to select, `0` to go back to the full mask.
- **Polyphony** ~ `CC_MONO`/`CC_POLY` switch a channel between monophonic and fully polyphonic (within what it may use) and silence held notes on the switch. `CC_SPLY` does the same without cutting off playing notes.
- **Steal behaviour** ~ with no free voices the oldest held note is stolen by default. This is a config tool setting, there is no CC.

Control Change reference

All CC numbers below are the **compiled-in defaults** (`repo/src/midi_defs.h`, `MIDI_DEFAULT_CCVALUES_INIT`). The CC map can be reconfigured (saved with the rest of the MIDI flash blob, see Saving your settings). If a CC below does nothing, check the CC map of your board.

Voice waveform and envelope

Each channel has one live template (waveform, envelope, pulse width) that is applied to each new note and pushed live to the notes held on the channel. A waveform or envelope CC change is heard on a held chord immediately.

CC	Name	What it does
20	`CC_NOIS`	Toggle the noise waveform bit
21	`CC_PULS`	Toggle the pulse waveform bit
22	`CC_SAWT`	Toggle the sawtooth waveform bit
23	`CC_TRIA`	Toggle the triangle waveform bit
24	`CC_TEST`	Toggle the test bit
25	`CC_RMOD`	Toggle ring modulation
26	`CC_SYNC`	Toggle hard sync
27	`CC_GATE`	Manual gate open/close on all voices the channel holds, separate from the automatic note-on/note-off gate. For holding a drone open
17	`CC_ATT`	Attack
18	`CC_DEC`	Decay

CC	Name	What it does
19	CC_SUS	Sustain
29	CC_REL	Release
1	CC_PWM	Pulse width (modulation wheel on most controllers), only audible with the pulse waveform active
28	CC_NOTE	One-shot manual pitch entry for the notes the channel holds

Waveform bits can be combined like on real SID hardware. Turning on more than one gives the thick, slightly metallic combined-waveform sound of a real chip.

Filter and volume

Filter and volume are chip wide registers ~ a CC in this section affects all SID chips in the channel mask at once, whatever number of notes is held.

CC	Name	What it does
32	CC_FFC	Filter cutoff frequency
33	CC_RES	Filter resonance
34	CC_FLT1	Route voice 1 through the filter
35	CC_FLT2	Route voice 2 through the filter
36	CC_FLT3	Route voice 3 through the filter
37	CC_FLTE	Route the external input through the filter
48	CC_3OFF	Disconnect voice 3 from the output entirely
49	CC_HPF	Enable the high-pass filter mode
50	CC_BPF	Enable the band-pass filter mode
51	CC_LPF	Enable the low-pass filter mode
7	CC_VOL	Channel volume

IMPORTANT On real SID hardware a voice routed into the filter (CC_FLT1-3/CC_FLTE) gives **no output** unless a filter mode (CC_HPF/CC_BPF/CC_LPF) is also enabled. That is how the chip behaves. If a sound goes silent after routing it through the filter, turn on a filter mode too.

SID / voice selection and channel mode

CC	Name	What it does
104	CC_SID1	Narrow this channel to SID 1 only (any nonzero value selects, 0 clears)
105	CC_SID2	Narrow this channel to SID 2 only

CC	Name	What it does
106	`CC_SID3`	Narrow this channel to SID 3 only
107	`CC_SID4`	Narrow this channel to SID 4 only
108	`CC_VCE1`	Narrow this channel to voice position 1 on every chip it can use
109	`CC_VCE2`	Narrow this channel to voice position 2
110	`CC_VCE3`	Narrow this channel to voice position 3
111	`CC_SPLY`	Toggle full polyphony for this channel without silencing held notes
119	`CC_GTEN`	Toggle automatic gate (gate follows note-on/note-off) - on by default
72	`CC_VELM`	Toggle velocity mode (velocity scales decay) - off by default
126	`CC_MONO`	Force this channel monophonic, silencing what it currently holds
127	`CC_POLY`	Force this channel fully polyphonic, silencing what it currently holds

Modulation and timing

One LFO, one arpeggiator and one portamento (glide) engine per channel, driven by a shared 1kHz tick. A loaded patch (see Patches) sets the same fields. A live CC change overwrites the patch values until the following Program Change.

CC	Name	What it does
0	`CC_LFOW`	LFO waveform: triangle, sawtooth, square, or sample & hold
2	`CC_LFOR`	LFO rate
3	`CC_LFOD`	LFO depth, `0` turns the LFO off (an idle LFO costs nothing)
4	`CC_LFOT`	LFO destination: pitch, pulse width, or filter cutoff
5	`CC_PORT`	Portamento (glide) time, `0` is off. A new note glides in from the previous note of the channel
6	`CC_ARPM`	Arpeggiator pattern: up, down, up/down, random, as-played, or table (see Arp tables)
10	`CC_ARPR`	Arpeggiator rate, free-running unless a MIDI clock is present, then read as a clock division
11	`CC_ARPO`	Arpeggiator octave range (repeats the held pattern up to 3 extra octaves), ignored in table mode, see Arp tables
12	`CC_ARPE`	Arpeggiator on/off for this channel, held notes feed the arpeggiator input pattern while on

CC	Name	What it does
30	`CC_ARPT`	Arp table select (0-15), the table slot the `CC_ARPM` table pattern reads. Needs table mode, see Arp tables

If PAL/PWM LFO pitch modulation is set as aftertouch target (`AT_VIBRATO`), channel pressure becomes a vibrato-on-pressure controller without touching `CC_LFOD`.

Second LFO

A second LFO of the same shape, stacked on top of the first. If both LFOs target the same destination their outputs add before the register write, two LFOs on pitch sum into one combined vibrato.

CC	Name	What it does
13	`CC_LFO2W`	LFO 2 waveform: triangle, sawtooth, square, or sample & hold
14	`CC_LFO2R`	LFO 2 rate
15	`CC_LFO2D`	LFO 2 depth, `0` turns LFO 2 off, independent of LFO 1
16	`CC_LFO2T`	LFO 2 destination: pitch, pulse width, or filter cutoff

Everything above for LFO 1 (idle at depth 0, patch and live CC interaction, `AT_VIBRATO`) applies to LFO 2 independently.

Unison

Stacks 3 oscillators onto one note for a thicker, detuned unison sound (MBSID Lead engine style). A SID chip has 3 oscillators: unison on a channel makes each note use all 3 voices of one chip, dropping that chip from 3-voice polyphony to monophonic. If no SID in the channel mask has all 3 voices free, the note is refused and nothing else is stolen.

CC	Name	What it does
41	`CC_UNIS`	Unison mode on/off for this channel (`>= 64` = on, `< 64` = off), only affects the **following** note-on
42	`CC_UDET`	Detune spread (0-127 range but takes the full 0-255 byte), `0` = no detune, higher values spread the outer two oscillators, up to roughly a semitone at full spread

The center oscillator plays the struck pitch, the other two are detuned up and down around it by the spread amount. Waveform, envelope and pulse width CCs and patch fields apply to all 3 oscillators together. They cannot get different waveforms or envelopes.

Panic and reset

CC	Name	What it does
120	CC_ASOF	All Sound Off, silences all held voices on all channels immediately
121	CC_RACT	Reset All Controllers ~ same effect as CC_ASOF
123	CC_ANOF	All Notes Off ~ same effect as CC_ASOF

These three are not channel-scoped. Any one of them on any channel silences the whole device, SID voices and FMOpl chip. Send one when starting a new song, in case a note was left stuck.

Cynthcart

Only relevant on a firmware build with _em (Cynthcart running through the C64 core of USBSID-Player, see USBSID-Pico-Cynthcart-manual). These three CC numbers are fixed and not part of the general CC map:

CC	Name	What it does
85	CC_CEN	Enable Cynthcart
86	CC_CDI	Disable Cynthcart
87	CC_CRE	Reset Cynthcart

While Cynthcart is active it takes over several CC numbers used elsewhere in this document (filter, envelope, tremolo, waveform selection, ...). See the Cynthcart manual for that CC list.

Deprecated

The old copy/link voice or SID family (CC_CVCE/8, CC_CSID/9, CC_LVCE/40, CC_LSID/56) is removed, no field is left in the CC map. CC numbers 8, 9, 40 and 56 are unassigned and silently ignored like any other CC without a handler.

Not wired

CC_BMSB/CC_BLSB (Bank Select MSB/LSB) and CC_MOD/CC_MODL (a second Modulation Wheel MSB/LSB pair, separate from CC_PWM) exist in the CC map but have no default number (0xFF/255, outside the valid 0-127 range) and no handler. Bank Select has nothing to select, Program Change reaches all 32 patches. Listed for completeness.

Patches

A patch is a saved timbre: waveform, envelope, pulse width, filter settings, LFO, arpeggiator and pitch bend range in one go. One MIDI Program Change (0xC0) selects it. Programs 0-31 map to the 32 patch slots, 32-127 do nothing (no Bank Select, Program Change reaches all 32).

Loading a patch changes the current template of the channel and pushes it live to the voices held on it, like a live CC change of the same fields. Routing (voice mask, polyphony, overrides) stays

untouched, a patch is a sound.

Factory patches

Patch 0 is the compiled-in channel default (plain triangle, full sustain), selecting it changes nothing audible. Patches 1-15 are a factory set to play on a fresh board:

Program	Instrument
1	Sawtooth lead
2	Pulse lead
3	Bass
4	Pluck
5	Organ
6	Filtered lead
7	Slow pad
8	Noise hit
9	Reed
10	Sync lead
11	Ring mod lead
12	Vibrato lead
13	Wah pad
14	Wide bass
15	Bright pluck

Patches 16-31 are for your own patches, zeroed (silent) until you save something into them.

Authoring your own patches over SysEx

Patches can be written, read back, saved and reset over MIDI without the config tool. All commands below use manufacturer ID `0x50`: `F0 50 <command> ⋯ F7`.

Byte	Command	What it does
`0x20`	`SYSEX_MIDI_PATCH_LOAD`	`F0 50 20 <slot> <48 nibbles> F7` ~ writes one patch into RAM. `<slot>` is 0-31.
`0x21`	`SYSEX_MIDI_PATCH_DUMP`	`F0 50 21 <slot> F7` ~ asks the device to send the current data of that patch back over MIDI (as a `0x22` message)
`0x22`	`SYSEX_MIDI_PATCH_DATA`	Device → host response to a `0x21` request, same 48-nibble shape as `0x20` input

Byte	Command	What it does
0x2A	SYSEX_MIDI_PATCH_SAVE	F0 50 2A <channel> <slot> F7 ~ the inverse of LOAD, captures the current live state of the given channel into a patch slot, RAM only
0x10	SYSEX_MIDI_SAVE	F0 50 10 F7 ~ persists all patches and the routing of every channel to flash
0x11	SYSEX_MIDI_LOAD	F0 50 11 F7 ~ reloads the last saved data (also runs at boot and on a MIDI System Reset)
0x12	SYSEX_MIDI_RESET	F0 50 12 F7 ~ erases all saved slots and resets all patches and channels to compiled defaults

Each patch field is sent as nibble bytes (values 0x00-0x0F, 7-bit safe), 48 nibbles per patch in this order: waveform+envelope+pulse-width template (5 fields), filter cutoff (16-bit, 4 nibbles), resonance, filter routing, filter mode, LFO 1 waveform/rate/depth/destination (4 fields), arpeggiator mode/rate/octaves (3 fields), pitch bend range, LFO 2 waveform/rate/depth/destination (4 fields) and unison on/off + detune spread (2 fields). That is 23 fields, 48 nibbles. An old 36-nibble dump (before LFO 2 and unison) is refused and logged. Re-export it from an up to date patch_editor.py.

SYSEX_MIDI_PATCH_LOAD only changes RAM, send SYSEX_MIDI_SAVE afterwards to keep it over a reboot. Loading a patch into slot 0-15 overwrites a factory instrument in RAM until the following SYSEX_MIDI_LOAD, reboot or SYSEX_MIDI_RESET. Slots 0-15 are pre-populated and can be overwritten.

The easiest way to build patches is patch-editor/patch_editor.py in this repository, a small graphical tool with a knob/slider/switch for each field above, Send/Read buttons and an importer for .s3i/.op2/.wopl instrument files (see its help text for FMOpl banks). It has a raw-hex box for pasting a SysEx line from anywhere, including patch-editor/midibox_sid_presets.md with roughly 500 MIDIbox SID V2 presets converted to the patch format of this firmware, ready to paste and try.

Arp tables

A GoatTracker/SidWizard-style arp/chord table. A table is a short authored sequence of semitone offsets relative to the held root note of the arpeggiator (the first key pressed), a custom arpeggio shape beside the 5 fixed shapes (up, down, up/down, random, as-played). Timing (free-running or MIDI-clock-synced), the CC_ARPR rate/division and CC_ARPE on/off work the same for fixed shapes and tables. Only the offset that follows differs.

There are 16 table slots with up to 16 steps each. Table 0 is a repeating major triad (0, +4, +7 semitones) as default. Tables 1-15 start empty (0 steps), an arpeggiator in table mode plays nothing with an empty table.

Using a table

Three things must line up for a table to play:

1. CC_ARPT picks the table slot (0-15) to read.
2. CC_ARPM set to "table" (top of its range, e.g. 127) picks table mode as pattern shape. CC_ARPT alone

does not switch the channel to table mode.

3. `CC_ARPE` turns the arpeggiator on for the channel.

`CC_ARPO` (octave range) is ignored in table mode, the steps of a table are fully authored.

Authoring tables over SysEx

Same manufacturer ID and LOAD/DUMP/DATA shape as patches and FMOpl patches:

Byte	Command	What it does
`0x27`	`SYSEX_MIDI_ARPTABLE_LOAD`	`F0 50 27 <slot> <36 nibbles> F7` ~ writes one table into RAM. `<slot>` is 0-15.
`0x28`	`SYSEX_MIDI_ARPTABLE_DUMP`	`F0 50 28 <slot> F7` ~ requests the current data of that table back
`0x29`	`SYSEX_MIDI_ARPTABLE_DATA`	Device → host response to a `0x28` request, also the `0x27` input

18 fields per table, each sent as two nibbles: 16 step offsets (signed, `-128` to `+127` semitones, as the two nibbles of the raw byte) followed by loop start and step count, 36 nibbles in total. `step_count 0` means the table is empty and the arpeggiator is silent in table mode. `loop_start` is the step the pattern jumps back to at the end, a table can have an intro before a repeating tail (e.g. steps 0-2 play once, then steps 1-2 loop with `loop_start = 1`, `step_count = 3`).

The "Arp Table" tab of `patch-editor/patch_editor.py` covers authoring visually, with a spinbox per step, loop-start/step-count sliders, Send/Read buttons and a one-click "Select on channel" action that sends `CC_ARPT` and sets `CC_ARPM` to table mode (still send `CC_ARPE` to turn the arpeggiator on).

Tables are saved/loaded/reset by the same `SYSEX_MIDI_SAVE`/`LOAD`/`RESET` as patches and channel routing, see Saving your settings.

Saving your settings

`SYSEX_MIDI_SAVE`/`SYSEX_MIDI_LOAD`/`SYSEX_MIDI_RESET` (see above) persist the routing of all channels, all patches, all arp tables and the CC map to a dedicated flash region, separate from the general board configuration. A firmware reflash with the same layout keeps it. A firmware upgrade that changes the size of the saved data (a new field) falls back to compiled defaults.

A MIDI System Reset (`0xFF`, a real-time status byte, no SysEx) reloads the last saved data, restoring your last saved sound. `SYSEX_MIDI_RESET` is the factory reset command.

Channel volume (`CC_VOL`) is never saved or restored by these commands, a client (MIDI player, ASID, your sequencer) is expected to set it. A System Reset or fresh boot comes back at a flat, audible default volume, all patch, routing and arp table settings come back as saved.

FMOpl

Some USBSID-Pico configurations have an FMOpl-capable chip installed beside or in place of a SID, either real OPL2 silicon or the software emulation of a clone chip (for example SIDKick-pico FMOpl emulation). It is one chip slot on your board, separate from the SID voice pool: 2-operator FM synthesis, 9 simultaneous notes and a small set of MIDI controls. This section only applies if your board has one configured, check with the config tool if you are not sure.

Targeting a channel at FMOpl

By default each MIDI channel targets a SID. `CC_FMEN` (CC 88) targets a channel at FMOpl: send `127` for FMOpl, `0` to go back to a SID. `CC_FMEN` overrides the SID voice mask/routing of the channel while on.

A channel needs `CC_FMEN` enabled before Program Change or notes on it reach the FMOpl chip. Notes on a channel without `CC_FMEN` set do nothing on the OPL chip. A channel reset to SID with `CC_FMEN 0` plays through the SID pool as normal.

`CC_FMEN` is sticky per-channel firmware state, it does not reset between songs. When using SID and FMOpl channels across pieces, reset each channel to the wanted target at the start of every song. See `patch-editor/strudel-demos/` for worked examples.

FMOpl patches

Program Change on an FMOpl-targeted channel selects one of 32 FMOpl patch slots, like the SID side but with a smaller patch shape: 2 FM operators (modulator and carrier) with multiplier, key-scale level/total-level, attack/decay, sustain/release and waveform each, plus a shared feedback/algorithm setting (classic FM/series or additive/parallel).

Slots 0-5 are compiled-in factory instruments:

Program	Instrument
0	Organ
1	Electric piano
2	Bass
3	Brass
4	Bell
5	Pad

The factory instruments are hand-designed from FM synthesis theory and copied from no commercial sound bank. Slots 6-31 start silent for your own patches.

Authoring works like the SID side, manufacturer `0x50`:

Byte	Command	What it does
0x23	SYSEX_FMOPL_PATCH_LOAD	F0 50 23 <slot> <22 nibbles> F7 ~ writes one FMOpl patch into RAM
0x24	SYSEX_FMOPL_PATCH_DUMP	F0 50 24 <slot> F7 ~ requests the data of that patch back
0x25	SYSEX_FMOPL_PATCH_DATA	Device → host response to a 0x24 request, also the 0x23 input
0x2B	SYSEX_FMOPL_PATCH_SAVE	F0 50 2B <channel> <slot> F7 ~ captures the current live FMOpl state of the given channel into a patch slot, RAM only

11 raw bytes per patch (2 operators x 5 fields plus one shared feedback/algorithm byte), 22 nibbles in total. These patches are not part of the SYSEX_MIDI_SAVE/LOAD/RESET flash persistence, they live in RAM and reset to the factory 6 on reboot. The FMOpl tab of patch-editor/patch_editor.py covers authoring like the SID side, including an importer for .s3i (Scream Tracker 3), .op2 (DMX GENMIDI, used by Doom/Heretic/Hexen) and .wopl (OPL3BankEditor/libADLMIDI) instrument files.

Live control

Three controls reach an FMOpl-targeted channel live, besides notes and Program Change:

CC	Name	What it does
7	CC_VOL	Channel volume, live, rewrites the loudness of all held notes immediately
1	CC_PWM	Modulation wheel. OPL2 has no pulse width, this drives a live **modulator-only** brightness/FM-intensity boost. Raising it makes the modulator louder relative to the carrier for a brighter, more overtone-rich tone without touching overall volume
-	Pitch Bend (0xE0)	Live, same as the SID side, bends all held notes on the channel using its bend range

All other CCs with meaning on a SID-targeted channel (waveform bits, ADSR, filter, LFO, arpeggiator, ...) do nothing on an FMOpl-targeted channel, no OPL equivalent is wired up yet.

Known limitations

- No LFO, arpeggiator or portamento on an FMOpl-targeted channel yet, only notes, Program Change, volume, mod wheel and pitch bend
- Velocity is accepted but not used yet
- A Program Change while a note is held does not change the timbre of that note (same as the SID side)
- FMOpl patches are RAM only (no flash save/load/reset yet)

A note on tuning

The clock rate that converts a MIDI note to chip pitch registers is hard to pin down. It depends on the chip/emulation in the FMOpl slot of your board and the compiled-in default is not confirmed on every configuration tested. If notes sound out of tune, the FMOpl tab of `patch-editor/patch_editor.py` has a live clock override with a held test note to dial it in by ear (see its help text). This is `SYSEX_FMOPL_SET_CLOCK` (`0x26`): `F0 50 26 <6 nibbles> F7`, a 3-byte (24-bit) big-endian clock value. An all-zero payload resets to the compiled-in default.

Development

More information about USBSID-Pico is available on the github repo

License

Software License ~ GNUv2

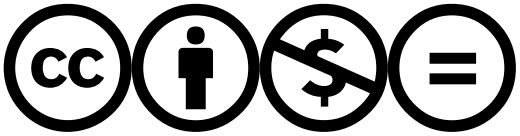
All code written by me in this repository is licensed under the terms of the GNU General Public License as published by the Free Software Foundation, version 2. Any code in this repository that is not written by me automatically falls under the licensing conditions by the authors of said code as mentioned in the source code header. For for more information about this license please read the LICENSE document in the root of the USBSID-Pico repository.

Configtool Software License ~ EPLv2

This program and the accompanying materials are made available under the terms of the Eclipse Public License 2.0 which is available at eclipse.org.
SPDX-License-Identifier: EPL-2.0

USBSID-Pico v1.5 PCB Hardware License ~ CC BY-NC-ND 4.0

USBSID-Pico v1.5 PCB © 2026 by LouD is licensed under
Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International



USBSID-Pico v1.3 PCB Hardware License ~ CC BY-SA 4.0

USBSID-Pico v1.3 PCB © 2025-2026 by LouD is licensed under
Creative Commons Attribution-ShareAlike 4.0 International



USBSID-Pico v1.0 PCB Hardware License ~ CC BY-SA 4.0

USBSID-Pico v1.0 PCB © 2024-2026 by LouD is licensed under
Creative Commons Attribution-ShareAlike 4.0 International



USBSID-Volume-daughterboard v1.0 PCB Hardware License ~ CC BY-SA 4.0

Volume-daughterboard v1.0 PCB © 2026 by LouD is licensed under
Creative Commons Attribution-ShareAlike 4.0 International



Author: LouD - generated on 2026-10-25